

Unit 0

Course Introduction

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROF. SUNDEEP RANGAN, SIDDHARTH GARG

Outline

 Course Admin

☐ What is Custom Hardware and What Can it Do?

Course Details

❑ Instructor

- Sundeep Rangan, 370 Jay St, 9-th floor

❑ Course assistant:

- Ruibin Chen,

❑ Office Hours:

- Instructor: Tuesday, 2-3

❑ Time and Location:

- Jacob's Hall, 6 Metrotech Room 201
- Wednesdays 11-1:30

Learning Objectives

- ❑ Identify tasks for hardware acceleration
- ❑ Design, simulate, and evaluate designs for hardware accelerators
 - Called **Intellectual Property** or **IPs**
 - Use state-of-the-art tools
 - High-level synthesis (HLS) and AI
- ❑ Integrate IP into processor-based systems for real-world applications
- ❑ Deploy and test systems on FPGA boards

Pre-Requisites

Anyone can learn hardware

- ❑ No prior hardware experience is required!
- ❑ No hard pre-requisites!
- ❑ Just have
 - Some software experience (e.g., C, python)
 - Desire to learn
- ❑ Many possible disciplines
 - Robotics, wireless, scientific computation, cryptography, networking, ...

Course Webpage

❑ All the material for the class is on GitHub

❑ Demos:

- SystemVerilog
- Vitis HLS
- Python

❑ Lecture notes

❑ Problems, Lab assignments

❑ Repository is work in progress

- Re-sync periodically

[Hardware Design](#) / Course Units

Course Units

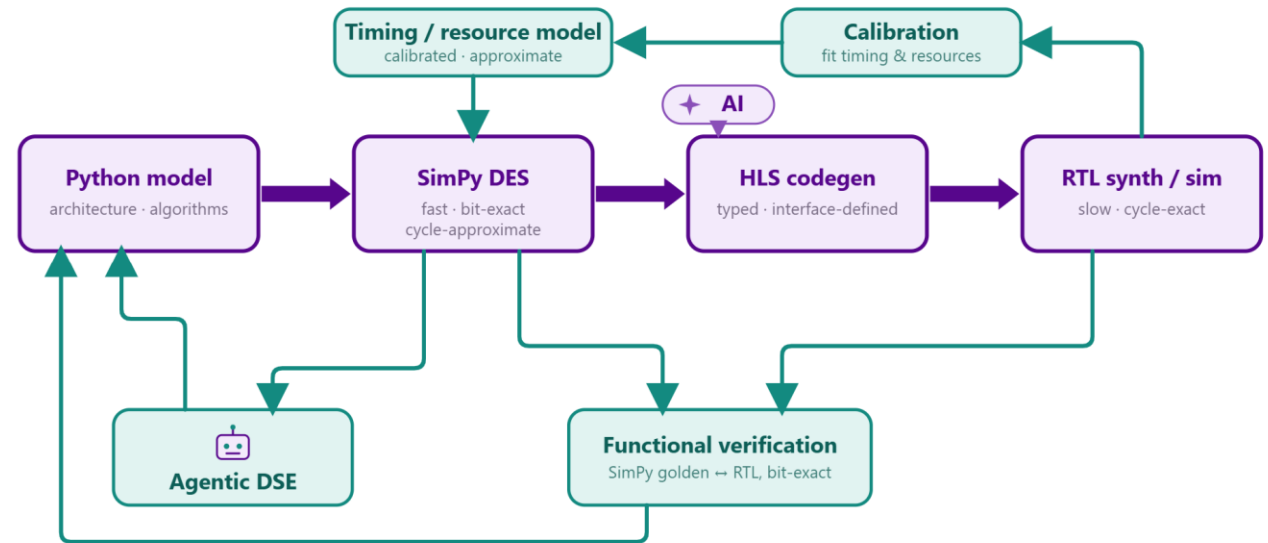
A tentative set of units is as follows. I've only added a few unit notes and problems.

- [Software and hardware set-up](#)
- [Unit 1: Basic digital Logic](#)
 - [SystemVerilog Examples: Simple scalar functions](#)
 - Lecture: [\[PDF\]](#) [\[pptx\]](#)
 - Problems: [\[PDF\]](#) [\[tex\]](#)
- [Unit 2: Bus basics and memory-mapped interfaces](#)
 - [Vitis HLS Example: Simple AXI4-Lite interface](#)

<https://sdrangan.github.io/hwdesign/docs>

WaveFlow

- ❑ Package for python-native circuit design
- ❑ We will use this package for:
 - Python-based AMD tool calls
 - Python golden models
 - Schemas and fixed point simulation
- ❑ You can also use it for your project



<https://sdrangan.github.io/waveflow/docs>

Software Set-Up: ECS Server

- ❑ Main tools are **Vitis** and **Vivado**
 - Basic software for digital design
- ❑ You can run all demos and labs on NYU ECS servers
 - No need for set-up on your local machine
- ❑ Ideal for MacOS users
 - Local set-up only supported on Windows and Linux
- ❑ Instructor will provide your email to NYU IT
- ❑ You will then be able to log into ecs03.poly.edu to ecs06.poly.edu
 - SSH for command line
 - Or FastX for a GUI-based system

```
[sr663@ecs03 ~]$ vitis --version
***** Vitis Development Environment
***** Vitis v2026.1 (64-bit)
**** SW Build 6497934 on 2026-06-16-10:22:29
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2026 Advanced Micro Devices, Inc. All Rights Reserved.
```

Software Set-Up: Local Machine

- ❑ You can also install Vitis and Vivado on local machine
- ❑ But only works for Windows and Linux
 - Does not work for MacOS
- ❑ Optional: Purchase an actual FPGA board
 - See next slide
- ❑ See installation instructions in class

[Hardware Design](#) / [Course Units](#) / [Getting Started](#)

Getting Started

Before stating the units, you will need to:

- [Install Vitis and Vivado](#): These are the tools used to synthesize and simulate the hardware
- [Clone the class GitHub repository](#): The repository contains the code for the course
- Purchase and [set-up and FPGA board](#): If you want to use a physical board, you will have to purchase a board and set it up. You can skip this step if you only want to simulate.

FPGA Boards

- ❑ Projects and labs can be deployed on actual hardware
 - Currently optional due to price
- ❑ We suggest: Pynq-Z2 board
 - \$300
 - USB connection to your laptop
 - Pynq: Simple python interface
 - Low-end, but ideal for teaching
- ❑ Other boards may be available in lab
 - Ex: RFSoc 4x2 for wireless communications



```
from pynq import Overlay  
overlay = Overlay("../scalar_fun_pynq
```

```
...  
# Input parameters  
a = np.random.randint(low=0, high=128)  
b = np.random.randint(low=0, high=128)  
  
# Sets the two inputs  
ip.register_map.a = a  
ip.register_map.b = b  
  
# Start the execution  
ip.register_map.CTRL.AP_START = 1
```

Tentative Grading

□ Points

- Homework + Labs: 25%
- Midterm: 25%
- Final: 25%
- Project: 25%

□ Labs will be mostly SystemVerilog and VitisHLS

- Approximately 4 Vitis HLS assignments
- Approximately 5 written assignments

□ Midterm and Final are in class

- Closed book, no electronic aids

AI Autograder

- ❑ An [LLM-automated grader](#)
- ❑ Will use for:
 - In class exercises
 - For Gradescope submissions
- ❑ Provides
 - Instant, detailed feedback
- ❑ But
 - May make mistakes
- ❑ Still experimental
 - Need your feedback

 **NYU** OpenAI Grader for NYU Applied Hardware Design
Profs. Sundeep Rangan, Siddharth Garg

Instructions for using this grader

Unit: Save Results Load Results

Question:

Upload latex solution: No file chosen

Question

Linear system: Consider the following SystemVerilog code:

```
always_ff @(posedge clk) begin
    x1 <= ((5*x1 + 3*x2)>>3);
    x2 <= ((4*x1 - 4*x2)>>3);
end
```

where >>3 denotes a right shift by 3 bits (division by 8), and all values are 32-bit signed integers.

(a) Suppose that $(x1, x2) = (8, 0)$, what is the value of

Your Solution

(a) $x1 = (5*8 + 3*0) \gg 3 = 45/8 = 5$
 $x2 = (4*8 - 4*0) \gg 3 = 4$

(b) One solution is:

```
def lin_sys(x1, x2, n):
```

Correct

Feedback

Your solution correctly computes the simulation function as required. Remember when updating in a system with parallel data type for arrays is important to ensure. Keep up the good work in carefully

Full Explanation

Part (a): The student correctly computed the formula and initial values (8,0). The student then updates the values by 8) and obtain $x1 = 5$ and $x2 = 4$, student provides a Python function with dtype np.int32, which is consistent with initial conditions in the first row and values of $x1$ and $x2$, applying the same logic by using previous values for updating the array X as required. The student's reference solution in both parts and passes.

OpenAI API Keys

- ❑ LLM Grader uses OpenAI models
- ❑ You will need your own OpenAI API key
- ❑ Follow [instructions](#)
- ❑ Cost is minimal:
 - For GPT-5.6 Luna, about \$0.30 for 1000 questions
- ❑ Note: Your OpenAI API key is kept privately
 - Stored on your browser

Using the OpenAI Models

OpenAI Keys

The grader is based on [OpenAI's API platform](#) which provides access to its powerful GPT models. To use the autograder, you will need to get an OpenAI API Key and register the key with the LLM grader:

- Go to the [OpenAI's API platform](#) page
- **Log In.** If you do not have an account, you will be asked to create one
- Go to the [API Key page](#) page.
- Create a OpenAI key.
- Go back to the Autograder webpage select **File->Preferences** and paste the key in the **OpenAI key** box.

Preferences

OpenAI API Key

Model:

Best for routine short-answer questions and single derivations — the cheapest at about \$0.30 per 1,000 graded questions.

Timeout (seconds):

Using AI

❑ It is encouraged!

- Use it for labs, problems, projects

❑ But learn to use it well:

- It still makes mistakes.
- Learn to test the results
- Write clear, specific prompts
- Use it as a tutor
- Have it generate questions
- Explain results

❑ On the midterm & final there are **no aids!**

Can you write SystemVerilog code to implement a linear function + ReLU:

$$y = \max(0, a \cdot x + b)$$

Sure! Here's a simple SystemVerilog module that computes $y = \max(0, a \cdot x + b)$ using combinational logic:

Systemverilog

```
module linear_relu #(
    parameter WIDTH = 16
)(
    input  logic signed [WIDTH-1:0] x,
    input  logic signed [WIDTH-1:0] a,
    input  logic signed [WIDTH-1:0] b,
    output logic signed [WIDTH-1:0] y
);

    logic signed [2*WIDTH-1:0] mult;
    logic signed [2*WIDTH-1:0] sum;

    always_comb begin
        mult = a * x;
        sum  = mult + b;
        y    = (sum > 0) ? sum[WIDTH-1:0] : '0;
    end

endmodule
```



Tentative Syllabus 1

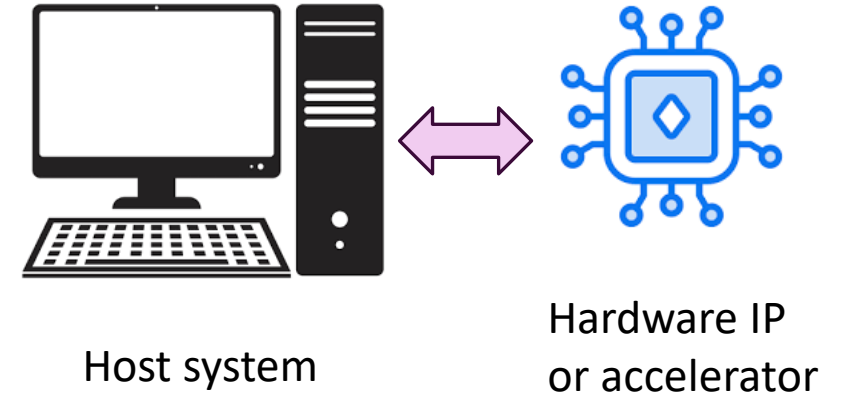
Date	Lecture
9/2/2026	Course Admin Unit 1: Data types and combinational logic
9/9/2026	Unit 2: Sequential logic and finite state machines
9/16/2026	Unit 3: Floating- and fixed-point arithmetic
9/23/2026	Unit 4: Bus basics and memory-mapped interfaces
9/30/2026	Unit 5: FIFO and streaming interfaces
10/7/2026	Midterm review
10/14/2026	Legislative Monday - no class
10/21/2026	Midterm

Tentative Syllabus 2

Date	Lecture
10/28/2026	Unit 6: Timing & critical path analysis
11/4/2026	Unit 7: Pipelining and loop optimization
11/11/2026	Unit 8: Shared memory architectures
11/18/2026	Unit 9: Systolic arrays
11/25/2026	Possible advanced topic
12/2/2026	Final exam
12/9/2026	Project presentations

Project

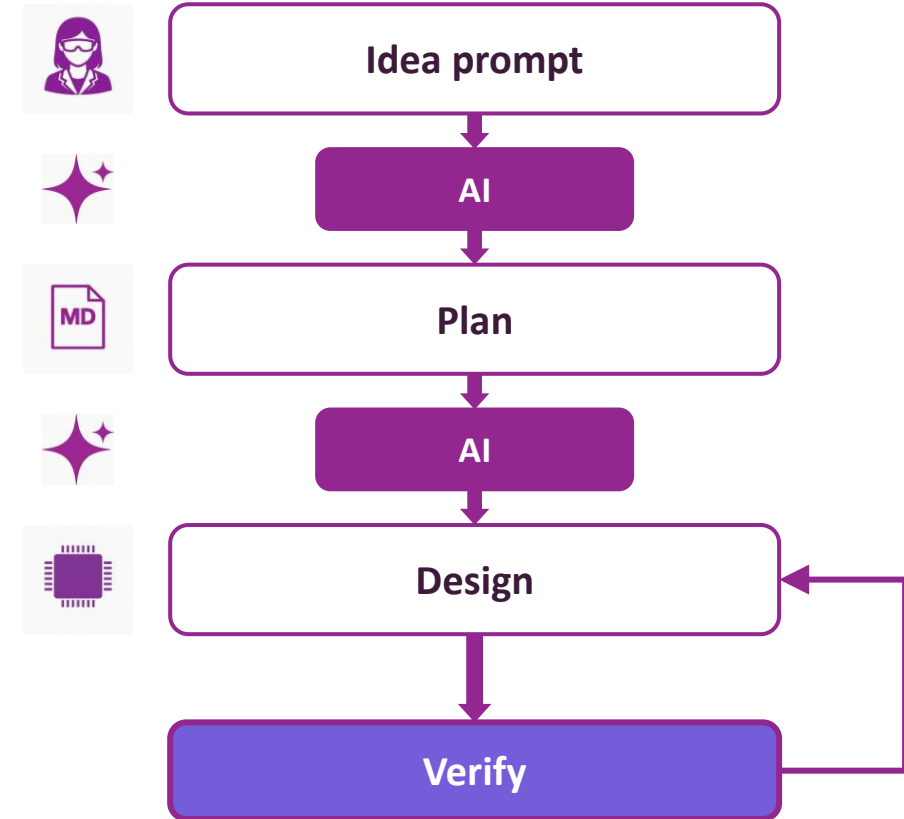
- ❑ Build your own IP
- ❑ You must provide:
 - A clearly defined hardware IP block (RTL or HLS)
 - A well-specified interface and integration model with a host system
 - A testbench and evaluation methodology
 - Documentation that allows others to reproduce your result
- ❑ Any topic of choice
 - Computer vision, communications, computing, ...
- ❑ Work in groups of 2 to 4
 - Individual projects require instructor permission



Use AI the Right Way!

AI works well only when...

- ❑ Ideas and goals are **precise**
- ❑ The design is **planned**
 - Context before AI fills in the detail
- ❑ Every output is **verified**
 - AI makes mistakes — a design needs proof
 - Know the expected result before you ask

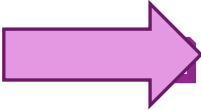


Project Grading

- ❑ See full rubric on [project page](#) on the GitHub pages
- ❑ Submission on GitHub
 - Create a repo for the project. You can start now!
 - Submit only the repo URL.
- ❑ IP definition and interface: Clear role, well-justified boundary between host and IP
- ❑ IP design: Efficient design, correct, design choices
- ❑ Verification and evaluation:
 - Testbench that are comprehensive, test corner cases, evaluate performance
 - Measure design options
- ❑ Organization and documentation: Assume you are pitching to a prospective employer

Outline

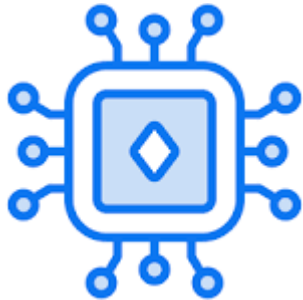
□ Course Admin

 What is Custom Hardware and What Can it Do?

Hardware vs. Software

Hardware

Any electronic circuit that performs some computation



Software

Instructions executed on hardware



Hardware Comes in Many Forms



General purpose CPU

Intel, AMD, ARM



DSP

Texas
Instruments,
Qualcomm
Hexagon



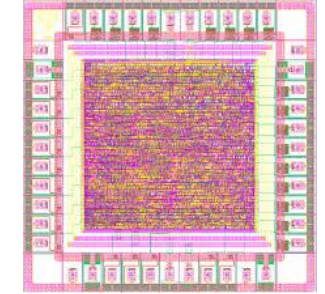
GPU

Nvidia
Intel
Radeon



FPGA

Xilinx / AMD,
Intel / Altera



Custom ASIC

Qualcomm,
Apple,
Broadcomm

General Purpose vs. Custom



General purpose CPU



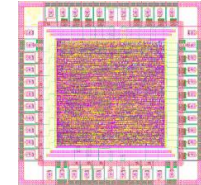
DSP



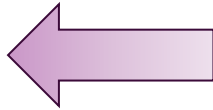
GPU



FPGA



Custom ASIC



General purpose,
Applies in many domains,
Re-programmed with software,
Performance via SW optimization, multi-cores



Domain specific,
HW optimized for specific task,
Can enable:

- Higher performance,
- Lower power / OP, size

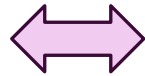
Focus of this course

Accelerator or IP Model



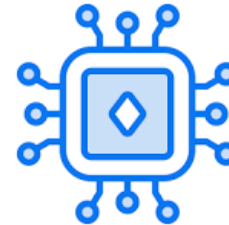
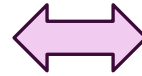
Host system

Runs main application,
User interface,
May be embedded as well
(e.g., smartphone)



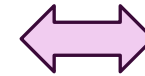
Embedded Processor

(DSP, ARM,...)
Configures HW,
Manages data transfer,
Performs some DSP



Hardware IP or accelerator

Compute intensive tasks
Domain specific



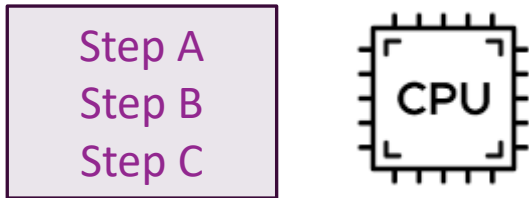
High-speed IO

Network interface,
Digital communication data

This class

What HW Does Well: Parallelism

Single thread processor

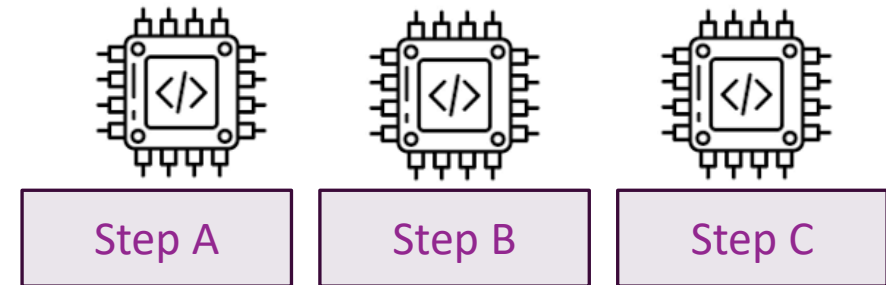


Throughput

$\propto$ Clock frequency

$\propto$ *Power*²

Custom hardware



Throughput

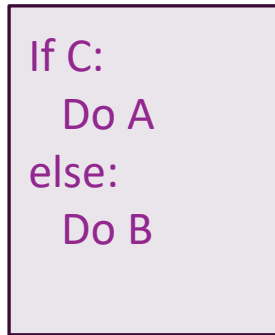
$\propto$ Hardware instance

$\propto$ *Power*

HW parallelism can be fundamentally more efficient

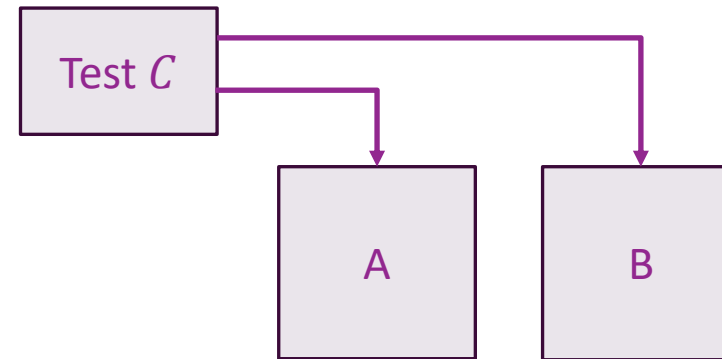
What is HW is Bad at: Conditional Logic

Single thread processor



Conditional / intermittent logic: **good**
Use all resources to whatever is needed

Custom hardware



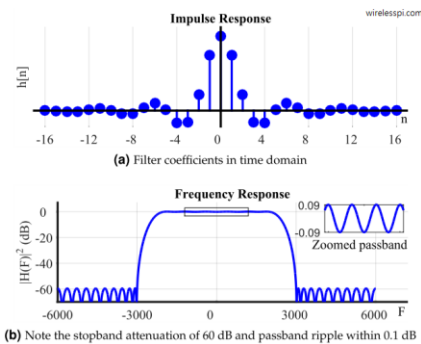
Conditional / intermittent logic: **poorly**
Need logic for all paths
Paths not used are idle

Example Accelerator: Filtering in DSP

$$y_n = b_0x_n + b_1x_{n-1} + \cdots + b_Kx_{n-K}$$

CPU / DSP:

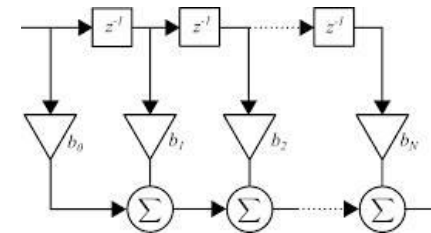
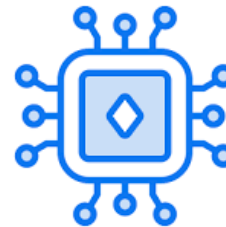
Design taps
Intermittent,
delay insensitive



$b_0, \dots, b_K$

FPGA / ASIC accelerator

Run filter
High-speed, repeated, high-parallelism



What is Custom Hardware Good At

❑ Compute-intensive kernels

- High arithmetic density, many operations per data

❑ Structured, repetitive tasks

- Same operation repeated many times
- Regular data flow, memory access

❑ Regular dataflow and memory patterns

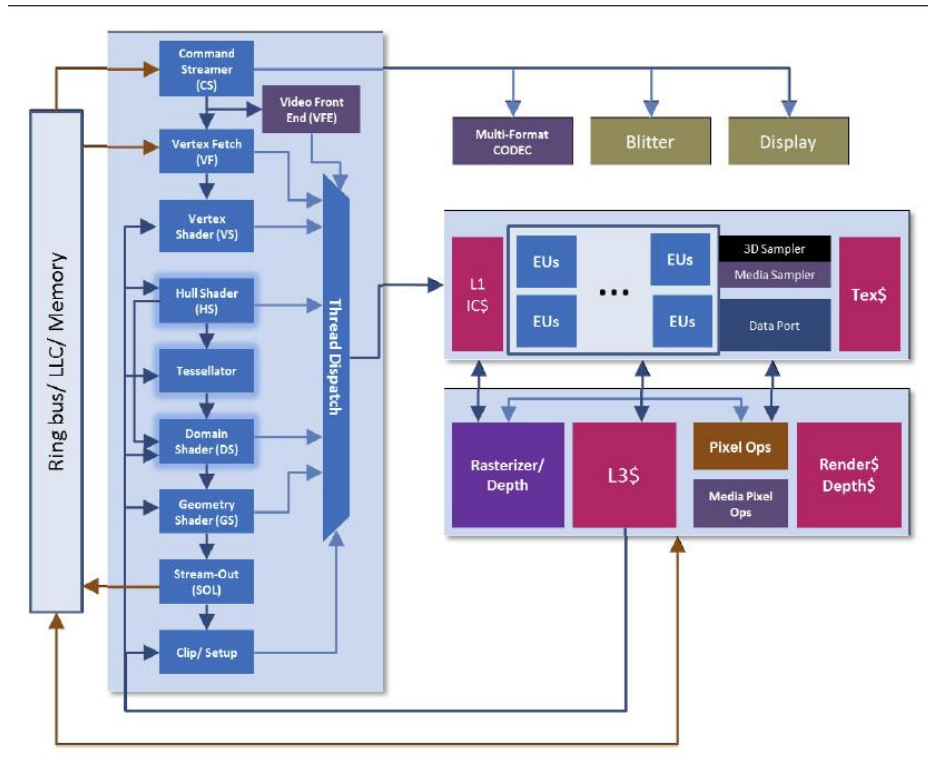
- Tight control of timing, no OS jitter

❑ High throughput with low energy

- Custom datapaths, optimized bit-widths, specialized memories

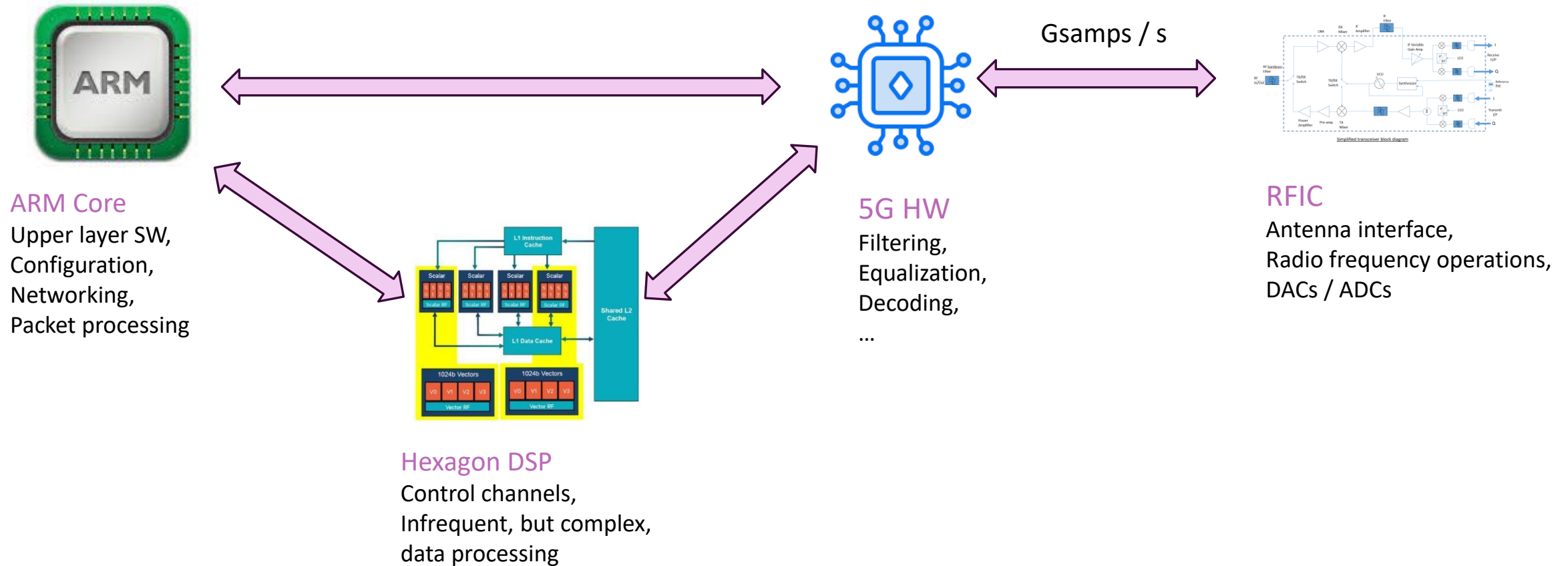
❑ Examples: DSP, linear algebra, ML inference, crypto, compression

Example: Intel Quick Sync Media Processor



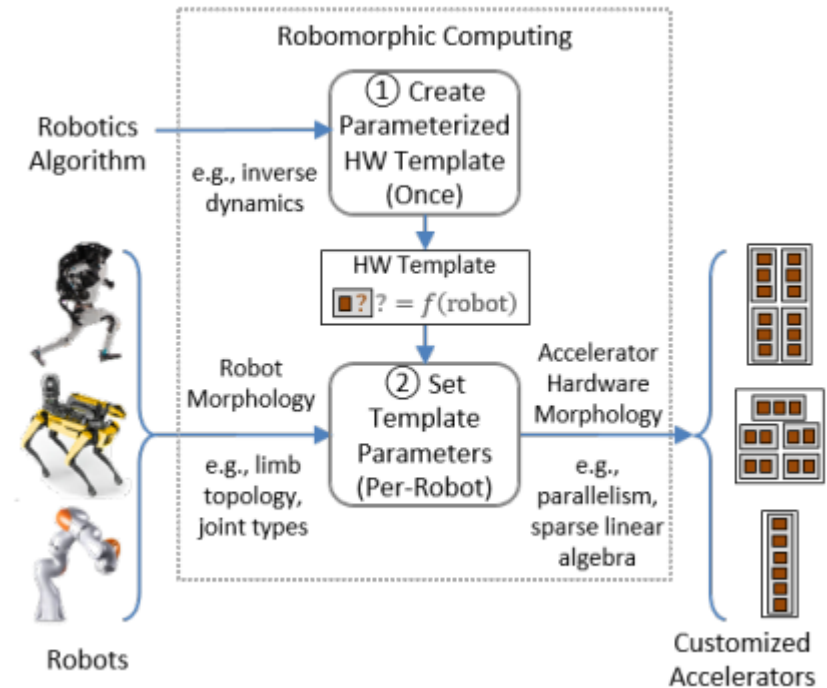
- ❑ Intel specialized graphics units
 - Programmable shaders for parallel math
 - Fixed-function blocks (rasterizer, tessellator, texture units)
 - Media engines for video encode/decode (Quick Sync)
- ❑ Integrated into Core-i3 / i5 / i7 / i9 processors
 - Physically part of the CPU die
 - Shares memory hierarchy with CPU cores
 - High-speed on-chip interfaces
- ❑ Direct access to system memory
 - High-bandwidth internal fabric
 - Connecting CPU, GPU, and media engines
 - Low-latency paths

Example: 5G Modem Core



Example: Robomorphic Computing

- ❑ Custom accelerator for robotic computation
- ❑ Performs complex kinematic simulations for:
 - Model predictive control
 - Path planning
- ❑ Key challenge:
 - How do you describe robot morphology?
 - Need a general structure applicable to many robots
 - Many algorithms



Neuman, S. M., Plancher, B., Bourgeat, T., Tambe, T., Devadas, S., & Reddi, V. J. (2021, April). Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (pp. 674-686).

FPGA vs. ASIC

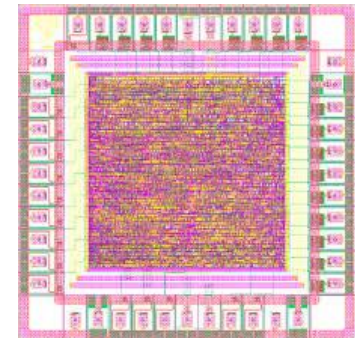
❑ FPGA: Field-Programmable Gate Array

- Reconfigurable hardware
- Great for prototyping and teaching: no fabrication needed
- Slower and less power-efficient than ASICs
- Research, rapid development, low-volume products, networking gear, some accelerators



❑ ASIC: Application-Specific Integrated Circuit

- Custom hardware chip designed for one purpose (e.g., video codec, 5G modem, ML accelerator)
- Much faster and more power-efficient than FPGAs
- Cannot be changed after fabrication — the design is permanent
- Manufactured (“fabricated”) at a semiconductor foundry
- Ex: TSMC,
- Smartphones, GPUs, Automotive, ...



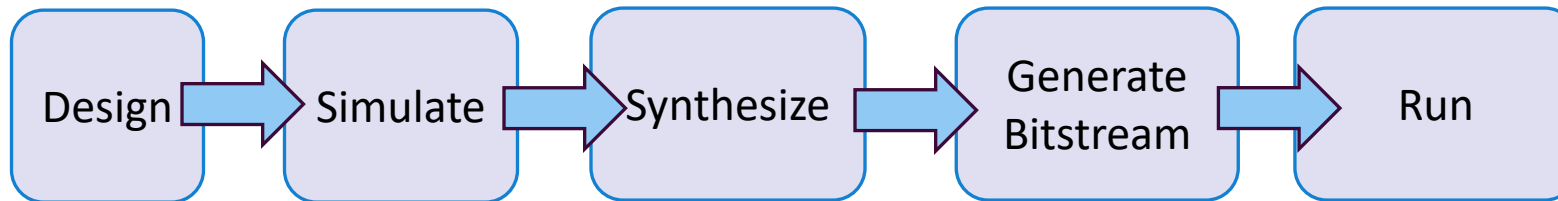
Turnaround Time Contrasted

Seconds

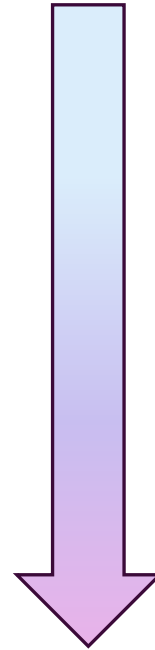
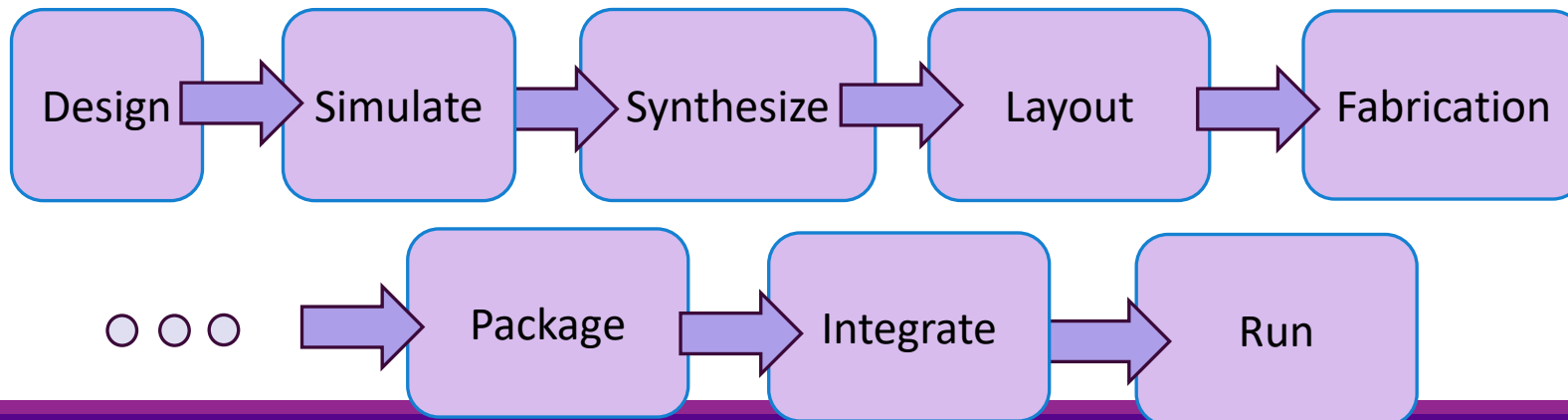
Software



FPGA



Custom ASIC



Months

Three Conceptual Challenges for Hardware

❑ Concurrency

- Hardware modules run in parallel
- Like multi-threaded programs
- Must consider resource contention, information passing, ...

❑ Long turnaround time

- Need reliable simulation and test before fabrication
- High level of rigor

❑ Need for interface abstraction

- Hardware must work over broad range of problems
- But provide gains specific to domain

HW Design Is Getting Easier

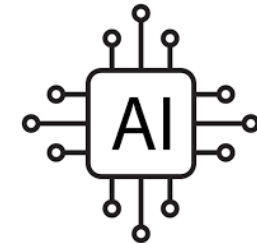
Traditional view: HW is Hard

- ❑ Rarely-used languages
- ❑ Small, elite community
- ❑ Complex tool chain
- ❑ Long turnaround time
- ❑ Need to design concurrent systems



New Tools

- ❑ High-Level Synthesis
 - Program in C/C++/Python
- ❑ AI-assisted design
 - LLMs are learning HW fast
- ❑ Python-based tool flow
 - FPGA Python APIs
 - Python simulation, ...



In-Class Exercise

Question 1. HW mapping

In this in-class exercise, you will brainstorm about what mathematical or computational operation could benefit from custom hardware implementation. The operation could be any field of your choice, such as image processing, machine learning, or cryptography.

- a. What is a mathematical operation that could, in part, be mapped to custom hardware?
- b. Which part of the computation could be accelerated by custom hardware? Why? Remember that hardware acceleration is most beneficial for computationally intensive tasks that can be parallelized and repeated.
- c. Which part of the computation is best left to software? Why? Remember that software is more flexible, easier to modify than hardware and better for intermittent, variable tasks.

Later in the course, we will discuss how to implement the hardware and software portions of the computation and how different components can communicate. For now, just focus on identifying the operation and which parts are best suited for hardware or software.

☐ Break into groups

☐ Answer in IImgrader

In-Class Exercise on LLM Grader

- ☐ Break into groups
- ☐ Go to: <https://llmgrader-e6o7.onrender.com/>
- ☐ Select Unit 0
- ☐ Select Question HW mapping
- ☐ Iterate on your solution with your group

File **Edit** **Help** **Grade**

Unit: Unit 0: Introduc... **Question:** HW mapping **Part** All 

Question

In this in-class exercise, you will brainstorm about what mathematical or computational operation could benefit from custom hardware implementation. The operation could be any field of your choice, such as image processing, machine learning, or cryptography.

- What is the mathematical operation that could, in part, be mapped to custom hardware?
- Which part of the computation could be accelerated by custom hardware? Why? Remember that hardware acceleration is most beneficial for computationally intensive tasks that can be parallelized and repeated.
- Which part of the computation is best left to software? Why? Remember that software is more flexible, easier to modify than hardware and better for intermittent, variable tasks.

Your Solution

Type your answer here, paste an image, or attach with +